

Atividade Prática — Análise de Controlador de Interrupções – SD212_29_05_2026

1. Repositório para análise

A atividade será baseada no seguinte repositório público:

Repositório: https://github.com/adibis/Interrupt_Controller

O repositório apresenta um controlador de interrupções com 8 entradas, escrito em Verilog. Ele possui pastas separadas para código-fonte e testbench, além de um README explicando o funcionamento geral do projeto.

2. Contexto da atividade

Em um sistema embarcado, periféricos podem gerar eventos que precisam ser tratados pelo processador.

Uma interrupção é um mecanismo usado para avisar o processador de que um evento externo ocorreu, evitando que o software precise verificar continuamente o estado de todos os periféricos.

Nesta atividade, os alunos deverão analisar como um controlador de interrupções pode receber múltiplas fontes de interrupção e indicar ao processador que existe um evento pendente.

3. O que deve ser feito

- Acessar o repositório indicado.
- Ler o arquivo README do projeto.
- Identificar os arquivos presentes nas pastas src e testbench.
- Analisar os sinais de entrada e saída do controlador de interrupções.
- Entender como o controlador opera nos modos polling e custom priority.
- Interpretar o testbench fornecido no repositório.
- Elaborar um relatório respondendo às perguntas.

4. Conceitos que devem ser observados

Conceito	O que observar no repositório
Interrupção	Como o controlador recebe eventos externos e gera uma saída de interrupção.
Polling	Como o controlador verifica as fontes de interrupção no modo polling.
Prioridade	Como o modo custom priority altera a ordem de atendimento das fontes.
Processador e periférico	Como o controlador se comunica com o processador por sinais de controle e barramento.
Testbench	Como os estímulos simulam eventos de interrupção e confirmações do processador.

5. Perguntas orientadoras para o relatório

1. Qual é o objetivo geral do repositório analisado?
2. Quantas entradas de interrupção o controlador suporta?
3. Quais são os principais arquivos Verilog presentes na pasta src?
4. Quais arquivos estão presentes na pasta testbench?
5. Quais sinais representam as fontes de interrupção?
6. Qual sinal indica ao processador que existe uma interrupção ativa?
7. Como funciona o modo polling descrito no README?
8. Como funciona o modo custom priority?
9. Qual é a função do sinal de acknowledgement vindo do processador?
10. O que acontece após o reset do controlador?
11. Como o testbench estimula o controlador de interrupções?
12. O testbench cobre quais situações principais?
13. Qual a diferença entre polling e interrupção no contexto da aula?
14. Por que um controlador de interrupções é útil em um SoC?
15. O que seria necessário acrescentar para aproximar esse controlador de um sistema embarcado real?

6. Simulação

Os alunos deverão simular o projeto utilizando o testbench disponível no repositório. A simulação deve comprovar o funcionamento básico do controlador de interrupções e apresentar as formas de onda geradas.

Na análise, o aluno deverá identificar quais sinais são estimulados no testbench, quais situações são verificadas e como as formas de onda demonstram o comportamento esperado do controlador.

7. Entregáveis

- Relatório curto, de 2 a 4 páginas, respondendo às perguntas da atividade.
- Prints do repositório e trechos de código relevantes utilizados na análise.
- Evidência da simulação do projeto, contendo prints das formas de onda geradas no ModelSim, QuestaSim ou ferramenta equivalente.
- Explicação dos principais sinais observados na simulação, destacando as entradas de interrupção, o comportamento do controlador e a saída de interrupção gerada.
- Interpretação do testbench, indicando quais situações são estimuladas e quais comportamentos são validados.
- Explicação clara dos modos de operação *polling* e *custom priority*.